Java Backend Architecture

Interview Series

Chapter 15.5
CQRS Pattern

50+ Interview Q&A | Code Examples | Architecture Diagrams | Advanced Topics

Chapter 15.5 – CQRS Pattern

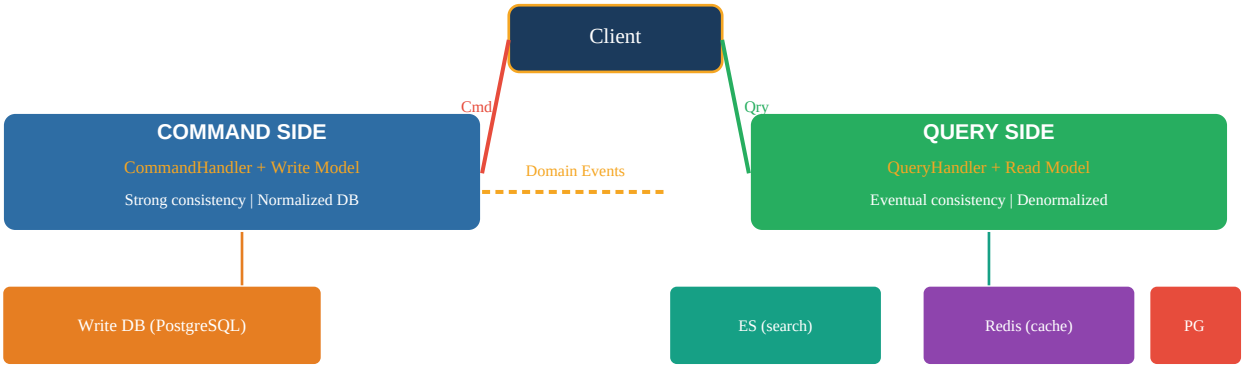
Introduction

Command Query Responsibility Segregation (CQRS) separates the write model (commands that change state) from the read model (queries that return data). This enables each side to be optimized independently: the write model for consistency and business rule enforcement; the read model for query performance using denormalized, pre-computed projections. CQRS is especially powerful combined with Event Sourcing and EDA.

Key Definitions

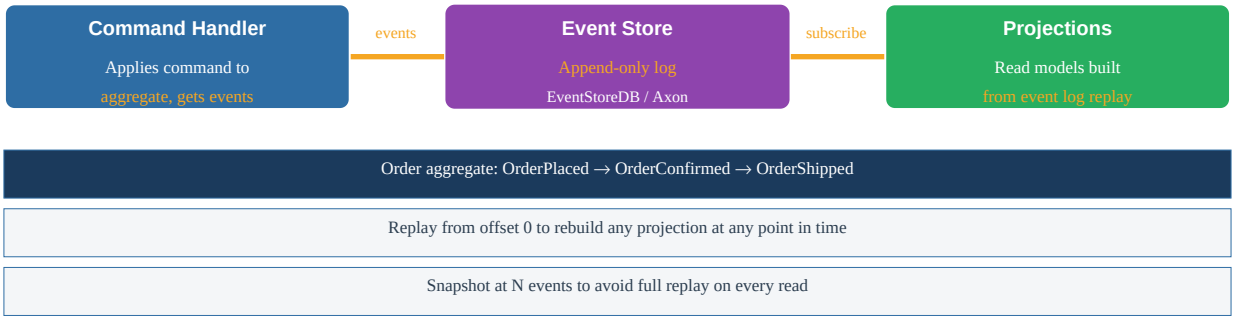
CQRS	Command Query Responsibility Segregation. Separate models for write (commands) and read (queries). Write model: normalized, consistent. Read model: denormalized, optimized per query pattern.
Command	An intent to change state. May be rejected. Imperative: PlaceOrder, CancelOrder, UpdateProfile. Handled by CommandHandler. Returns void or minimal acknowledgment.
Query	A request for data that does not change state. Returns a read model DTO. Can be served from denormalized read store (Elasticsearch, Redis) without touching write DB.
Read Model	Denormalized, pre-computed view of data optimized for specific query patterns. Can use different DB technology (ES for full-text search, Redis for leaderboards, PG for reporting).
Projection	A process that subscribes to domain events and updates the read model accordingly. One event can update multiple projections. Can be rebuilt by replaying events from offset 0.
Eventual Consistency	Read model reflects write model state after a short lag (Kafka processing time). During the lag, reads may return slightly stale data. Trade-off for read performance.
CDC (Change Data Capture)	Technology (Debezium) that captures every change in the source DB's transaction log and publishes to Kafka. Alternative to domain events for read model sync.
Dual-write Risk	Anti-pattern: writing to DB and Kafka separately. If app crashes between the two writes, they diverge. Prevented by outbox pattern or CDC.
Event Store	Append-only log of domain events (EventStoreDB, Axon). The source of truth in Event Sourcing. Current state computed by replaying events.
Snapshot	Periodic checkpoint of aggregate state stored to avoid replaying all events from the beginning. Read from latest snapshot + subsequent events to reconstruct current state.

CQRS Architecture – Command/Query Separation



Read Model Sync via Domain Events

CQRS with Event Sourcing Combined



CQRS vs Traditional CRUD Comparison

Aspect	Traditional CRUD	CQRS
Model	Single model for reads and writes	Separate write/read models
Read performance	Limited by normalized schema + JOINs	Optimized denormalized read models
Write consistency	Immediate	Immediate (write model)
Read consistency	Immediate (same DB)	Eventual (read model lag)

Complexity	Low	High – two models to maintain
Query flexibility	JOINs + indexes	Purpose-built per query pattern
Scaling	Shared read/write DB bottleneck	Scale read/write independently
Good for	CRUD apps, simple domains	Complex read requirements, high read:write ratio

Spring Boot CQRS – Command Side

```
// Command public record PlaceOrderCommand(String customerId, List<OrderItem> items) {} // Command Handler @Service @Transactional public class PlaceOrderCommandHandler { public OrderId handle(PlaceOrderCommand cmd) { // Validate + apply business rules Order order = Order.create(cmd.customerId(), cmd.items()); orderRepository.save(order); // Publish domain event via outbox outboxEventPublisher.publish(new OrderPlacedEvent(order)); return order.getId(); } } // REST controller – command endpoint @PostMapping("/orders") public ResponseEntity<Map<String, String>> placeOrder( @Valid @RequestBody PlaceOrderRequest req) { PlaceOrderCommand cmd = new PlaceOrderCommand(req.customerId(), req.items()); OrderId id = placeOrderHandler.handle(cmd); return ResponseEntity.created(URI.create("/orders/" + id)) .body(Map.of("orderId", id.value())); }
```

Spring Boot CQRS – Query Side with Elasticsearch

```
// Read model – denormalized for search @Document(indexName = "orders") public class OrderSearchView { @Id private String orderId; private String customerId; private String customerName; // Denormalized from User service private String status; private List<String> productNames; // Denormalized from Product service private BigDecimal totalAmount; private Instant createdAt; } // Query handler – serves from Elasticsearch (no write DB touch) @Service public class OrderQueryService { @Autowired ElasticsearchOperations esOps; public Page<OrderSearchView> searchOrders(OrderSearchQuery query) { var criteria = new Criteria("customerName").contains(query.name()) .and(new Criteria("status").is(query.status())); return esOps.search(new CriteriaQuery(criteria), OrderSearchView.class, ...); } } // Projection – subscribes to Kafka, updates read model @KafkaListener(topics = "order.placed") public void onOrderPlaced(OrderPlacedEvent event) { // Enrich with customer name from User service (or from event if ECST) String customerName = userService.getName(event.customerId()); OrderSearchView view = new OrderSearchView(event.orderId(), customerName, ...); esOps.save(view); }
```

Read Model Synchronization Strategies

Strategy	How It Works	Pros	Cons
Domain Events via Kafka	CommandHandler publishes event; projection subscribes	Explicit, auditable	Dual-write risk without outbox
Outbox + CDC	Event written to outbox table; Debezium reads WAL and publishes	Atomic, reliable	Infrastructure overhead
Scheduled Polling	Projection polls write DB on schedule	Simple	High lag, DB load, not real-time
Synchronous Dual-write	Service writes to both write DB and read DB directly	Simple	HIGH RISK: inconsistency on failure

Interview Q&A;

Q1. What does CQRS stand for and what is its core idea?

Command Query Responsibility Segregation. Core idea: separate the model used for writes (commands) from the model used for reads (queries). Write model enforces business rules with strong consistency. Read model is denormalized and optimized for specific query patterns. They sync asynchronously via domain

events.

Q2. What problem does CQRS solve?

In traditional CRUD: the same normalized relational model is used for both reads and writes. Reads require complex JOINs that conflict with write optimization. CQRS solves: (1) Read performance—denormalized read models serve queries without JOINs, (2) Scaling reads and writes independently, (3) Using different DB technology for reads vs. writes.

Q3. What is eventual consistency in CQRS?

After a command updates the write model, the read model is updated asynchronously (via Kafka events). During the propagation lag (milliseconds to seconds), queries may return stale data. This is the fundamental trade-off: read performance vs. immediate consistency. Design: inform users ('updated just now, data may take a moment to refresh'), or use optimistic UI updates.

Q4. When should you use CQRS?

Use CQRS when: (1) Read:write ratio is very high (reads dominate), (2) Query patterns are diverse (search, aggregate, filter by many dimensions), (3) Domain is complex with many business rules on the write side, (4) Different scaling needs for reads vs. writes. Avoid for simple CRUD apps—adds unnecessary complexity.

Q5. What is a projection in CQRS?

A projection subscribes to domain events and maintains a read model that reflects the current state. Example: OrderProjection subscribes to OrderPlaced, OrderConfirmed, OrderShipped events and maintains an Elasticsearch index of orders. A projection can be rebuilt from scratch by replaying events from offset 0. Multiple projections can serve different query patterns from the same events.

Q6. What is the dual-write risk in CQRS?

Anti-pattern: application writes to the write DB AND separately writes to Kafka/read model. If the app crashes after the DB write but before the Kafka publish, the read model is out of sync. Prevention: use the outbox pattern (write event to outbox table in same transaction) or CDC (Debezium reads DB transaction log and publishes—no application-level dual-write).

Q7. How do you rebuild a read model projection?

Stop the projection consumer. Reset its consumer group offset to 0 (beginning of topic). Optionally truncate the read model store. Restart consumer—it replays all events from the beginning and rebuilds the read model. This requires Kafka topic retention to be long enough. Can be done without downtime by building new projection in parallel and swapping on completion.

Q8. What DB technologies are suitable for CQRS read models?

Depends on query pattern: Elasticsearch for full-text search and complex filtering. Redis for leaderboards, counters, real-time data. MongoDB for flexible document queries. Apache Cassandra for time-series queries with predictable patterns. PostgreSQL materialized views for complex reporting. The key: choose the DB that best fits the specific read query, not a one-size-fits-all.

Q9. How does CQRS relate to Event Sourcing?

They are separate patterns but combine naturally. Event Sourcing: store state as events rather than current state. CQRS: separate read and write models. Combined: Event Store is the write side (append events). Projections replay event log to build read models. Axon Framework and EventStoreDB support this combination natively in Java.

Q10. What is Command Query Separation (CQS) vs CQRS?

CQS (Bertrand Meyer): a method should either change state (command, return void) or return data (query, no side effects). Applied at method level: userService.update() vs userService.find(). CQRS: same principle applied at architectural level—separate object models, potentially separate services/databases. CQRS is CQS applied to the whole system architecture.

Q11. Describe the write side of CQRS in Spring Boot.

Command object (POJO/record): PlaceOrderCommand. CommandHandler (@Service, @Transactional): validates, applies domain logic, saves to normalized write DB, publishes domain event. Returns minimal result (ID only). The write side has no concept of read models—it deals only with business rules and persistence.

Q12. Describe the read side of CQRS in Spring Boot.

QueryObject: OrderSearchQuery with filter criteria. QueryHandler (@Service): queries the read model store (Elasticsearch, Redis). Returns rich DTO with denormalized data (order + customer name + product names). No business logic—pure data retrieval. Can use different DB than write side. REST controller calls QueryHandler directly, never CommandHandler.

Q13. What is optimistic UI in the context of CQRS eventual consistency?

When the command is accepted, the UI immediately shows the expected result without waiting for the read model to update. Example: user places order → UI immediately shows 'Order placed' and adds it to the order list using local state, even before the Elasticsearch read model has updated. If the command is rejected, the UI rolls back the optimistic update.

Q14. How does CDC (Debezium) help with CQRS read model sync?

Debezium reads the source DB's transaction log (PostgreSQL WAL). Every INSERT/UPDATE/DELETE on the write model tables generates a Kafka event. The read model projection subscribes to these CDC events and updates its store. No application-level event publishing needed—CDC captures changes automatically. Advantage: no outbox table needed; DB changes always captured regardless of which application made them.

Q15. What are the failure modes in CQRS read model sync?

(1) Consumer failure: Kafka retains messages; consumer catches up on restart. (2) Projection bug: rebuild from event replay after fix. (3) Schema mismatch: event format changed without consumer update. Prevention: contract testing. (4) Read store unavailable: queries fail; write operations still work (read/write independently scalable). Alert on consumer lag and Dead Letter Queue.

Q16. How do you handle the fact that CQRS queries may return stale data?

(1) Document the eventual consistency to clients—SLA on max lag. (2) Optimistic UI updates—show expected state immediately. (3) Include version/timestamp in read model—clients can detect staleness. (4) For critical reads (payment confirmation): query write model directly (bypass read model for that one case). (5) Accept the trade-off—for most queries, millisecond lag is imperceptible.

Q17. What is read model enrichment?

The read model contains data from multiple sources, denormalized for the query. Example: OrderSearchView contains order data + customer name (from User service) + product names (from Product service). The projection enriches the event data by calling other services or reading from their published events. Trade-off: enrichment adds latency to projection updates; pre-computed data speeds up queries.

Q18. Explain projection idempotency.

Projections must be idempotent: processing the same event twice must not corrupt the read model. Example: if OrderPlaced is processed twice, the read model should have exactly one entry, not two. Implementation:

use event ID as idempotency key. Or: use upsert operations (INSERT ... ON CONFLICT DO UPDATE) instead of blind inserts.

Q19. What is CQRS without Event Sourcing?

Most common CQRS implementation: write side uses traditional relational DB. Domain events are published after successful transactions (via outbox). Projections consume events and update read models (Elasticsearch, Redis). Full Event Sourcing (append-only event log as source of truth) is optional and adds complexity. Many teams successfully use CQRS with regular DB + event publishing.

Q20. How does CQRS improve read scalability?

Read models are read-only (no writes) and can be cached aggressively. Can add read replicas without write complexity. Can use in-memory stores (Redis) for hot data. Can distribute read models across regions closer to users (CDN for static read models). Write model doesn't compete with reads for DB connections and I/O.

Q21. What are the CQRS trade-offs compared to simple CRUD?

CQRS adds: (1) Two code models to maintain (write + read), (2) Sync infrastructure (Kafka, CDC, projections), (3) Eventual consistency to explain to business stakeholders, (4) Infrastructure overhead (Elasticsearch, Redis for read stores), (5) More complex debugging (trace event through write → Kafka → projection). Justified when: complex query requirements, high read load, diverse query patterns.

Q22. How do you test CQRS projections?

Unit: given a sequence of events, assert that the projection produces the expected read model state. Integration (Testcontainers): publish event to Kafka, wait for projection to consume, query read model. Contract: verify event schema matches what projection expects (Pact consumer contract). Rebuild test: reset projection to offset 0, replay all test events, assert final state.

Q23. What is a materialized view vs a CQRS projection?

Materialized view: DB feature—pre-computed view stored on disk, refreshed on schedule or trigger. Stays within the same DB technology. Limited to SQL-expressible computations. CQRS projection: application code consumes events and updates a separate read store. More flexible: can use different DB technology, custom logic, external data enrichment. Materialized views are simpler for small scale; CQRS projections for complex/polyglot scenarios.

Q24. How does Axon Framework support CQRS + Event Sourcing in Java?

Axon provides: @CommandHandler annotation on aggregate methods, @EventSourcingHandler for event application, @QueryHandler for query handling, @EventHandler for projections. CommandGateway and QueryGateway for dispatching. Axon Server as event store and message routing. Handles command routing, event storage, projection rebuilding out of the box.

Q25. What is the read model for a 'user dashboard' query in CQRS?

Traditional: JOIN users, orders, profiles, settings = complex SQL, slow. CQRS read model: UserDashboardView { userId, name, email, orderCount, lastOrderDate, subscriptionStatus }. Pre-computed and stored in Redis. Updated by projections listening to UserUpdated, OrderPlaced, SubscriptionChanged events. Dashboard query = single key lookup in Redis = microseconds.

Q26. How do you handle 'undo' operations in CQRS?

CQRS with event sourcing: naturally supports undo by replaying events without the undone event. CQRS without event sourcing: publish a compensating command/event (CancelOrder), update write model, projection updates read model to reflect cancellation. No true undo of DB state—instead, create new state that represents the undone action.

Q27. What is the cost of projection rebuilds?

Rebuilding a projection replays all historical events from Kafka. For large datasets (millions of events), this can take hours. Optimizations: Kafka topic compaction (keep only latest state per key), periodic snapshots of read model state, parallel replay with increased consumer threads, rebuild into new index and swap alias (zero-downtime rebuild in Elasticsearch).

Q28. How do you handle tenant isolation in a multi-tenant CQRS system?

Include tenant ID in all events. Partition Kafka topics by tenant ID for ordering. Read models have tenant_id column with row-level security. Projections filter events by tenant ID when updating read models. For strict isolation: separate read model indexes/tables per tenant.

Q29. Describe a CQRS implementation for a real-time leaderboard.

Write model: GameResultRepository (PostgreSQL). Each game result saved normally. On GameResultSaved event: projection subscribes, uses Redis ZADD to update sorted set leaderboard. Query: ZREVRANGE leaderboard:game123 0 9 = top 10 players in O(log N). No SQL JOIN or GROUP BY needed. Real-time updates as projections process events. This is a perfect CQRS use case.

Q30. What is the relationship between CQRS and microservices?

CQRS can be applied within a single microservice (most common) or across services. Within a service: separate command and query models in same codebase, different DB tables. Across services: dedicated read-model service consumes events from write-model service. Not all microservices need CQRS—only services with complex read requirements or high read load.

Q31. How do you ensure no data loss in CQRS projection sync?

Outbox pattern: events written to DB in same transaction—never lost. Kafka durability: replication factor 3, acks=all. Consumer offset management: commit offset only after successful projection update. Dead Letter Queue: failed events captured for replay. Monitoring: alert on consumer lag > threshold, DLQ size > 0.

Q32. What happens to the read model when the write model schema changes?

Read model is independent—it defines its own schema for its query needs. If the event schema changes: update the projection to handle both old and new event formats. If the read model schema needs to change: rebuild projection from events (reads new format going forward, old events produce old read model fields until rebuilt). Schema Registry prevents incompatible events from reaching projections.

Q33. What is write-through caching vs CQRS read model?

Write-through cache: update cache and DB synchronously on every write. Simple but couples write performance to cache update latency. CQRS read model: updated asynchronously via events. More complex but: decoupled write/read performance, supports different DB technologies, can be rebuilt independently, supports complex transformations write-through cache cannot.

Q34. How does CQRS handle complex aggregation queries?

Traditional: SELECT SUM(amount), COUNT(*), AVG(amount) FROM orders GROUP BY customer_id = expensive query. CQRS: projection maintains CustomerOrderStats { customerId, totalSpent, orderCount, avgOrderValue }. Updated incrementally on each OrderPlaced event: totalSpent += amount, orderCount++, recalculate avg. Query = single row lookup. Much faster for high-frequency analytics.

Q35. Describe monitoring for a CQRS system.

Write side: command processing latency, command failure rate, write DB connection pool, outbox queue depth. Sync: Kafka consumer lag per topic/partition, projection processing rate, DLQ size. Read side: query latency (p95/p99), read model store availability (ES/Redis health), cache hit rate. Business: read model freshness (time since last update per projection), data consistency checks between write and read models.

Q36. What are the benefits of CQRS for microservices evolution?

Read models can be changed without touching write model or other services. New read model for new query pattern: create new projection, replay events, deploy. Old read model still works until migrated. This is the 'open/closed' principle for data: closed for modification of write model, open for extension via new projections. Enables evolutionary architecture.